IS IT WORTH IT? · CALCULATOR 04B

Free Auto Web App

Architecture & implementation plan for evolving **“Drive There or MRT?”** into a free, automatic, install-on-your-phone web app — for review and clarification before any code is written.

◆ Draft v0.1 — for walkthrough & sign-off

Prepared for
withJia-tech

Date
23 August 2026

Repository
withJia-tech/Drive-or-mrt-in-sg

Baseline
Branch `claude/phone-location-detection-oqfi85` · PR #1

EXECUTIVE SUMMARY

The one-paragraph version

“Drive There or MRT?” is already a free, zero-backend web app: it runs entirely in the browser, hosts on GitHub Pages for $0, and calls nothing home. This plan turns it *automatic* — reduce the numbers you have to type down to near zero — while keeping it free and keeping the trait that makes it trustworthy: **it tells you every assumption it makes.** The location-detection feature just shipped (PR #1) is the first step. The rest is more of the same: auto-derive the real road distance, auto-suggest the destination, and auto-fill fares and parking rates from Singapore’s open data.

◆ THE ONE DECISION THAT SHAPES EVERYTHING ELSE

You used the word **“agent.”** This codebase was *deliberately* built with no agent — its own design notes call that “a finding, not an omission,” because the whole thing is ten arithmetic rules over numbers you supply. So before implementation, the central question to clarify is: **does “agent” mean (A) a smart, automatic deterministic app, or (B) an actual AI/LLM layer?** Section 7 lays out both, with a free and private option for each. My recommendation is A — but this is yours to decide, and the whole document is arranged so you can.

What you’re being asked to sign off

1. **The interpretation** of “free / auto / web app / agent” (Section 3).
2. **The target architecture** — what stays, what’s added (Section 5–6).
3. **The nine open decisions** in Section 10 — each has a recommended default, so signing off can be as fast as “defaults are fine.”

HOW TO WALK THROUGH THIS

Reading guide

This is a decision document, not a spec. Read Sections 3, 7 and 10 closely — that is where your input changes what gets built. Everything else is context so those choices make sense.

SECTION	WHAT IT SETTLES	NEEDS YOUR INPUT?
3 · Goal & interpretation	What the four words mean and what is out of scope	Confirm
4 · Where we are today	The current architecture and what just shipped	Context
5–6 · Target architecture	The layers, components and data flow of the auto app	Confirm
7 · The “agent” question	Deterministic vs. AI — the pivotal choice	Decide
8 · The auto upgrades	Routing, destination, open-data fares/parking, PWA	Decide
9 · Free-tier stack	Proof that every layer costs $0	Context
10 · Open decisions	The nine questions, each with a default	Decide
11–13 · Roadmap, risk, testing	Order of work, what could go wrong, how it’s proven	Context

3 · GOAL & INTERPRETATION

What “free auto web app” means here

The brief was four words. Here is how I’m reading each one, so we can correct any of them before building. If one of these is wrong, it is the cheapest possible moment to say so.

WORD	INTERPRETATION	CONCRETELY
Free	Zero running cost, forever. No paid APIs, no server bill, no database, no subscription — for you and for every user.	Static hosting on GitHub Pages ($0). Only free-tier or open data. No secret keys that could rack up a bill.
Auto	The app fills in what it can, so the user types as little as possible — ideally just “how long will you park?”.	Auto-detect location (done), auto road distance, auto destination suggestion, auto fares/parking from open data.
Web app	Not just a page — an installable app that works on a phone at the door, even offline.	A Progressive Web App (PWA): “Add to Home Screen”, an icon, offline support via a service worker.
Agent	*Ambiguous — see Section 7.* Either “a smart automatic app” or “an AI/LLM does the thinking.”	Recommended reading: automation, not autonomy. But a free, private AI option exists and is laid out for you.

IN SCOPE

- Cutting manual input to near zero
- Real road distance, not straight-line
- Open-data parking & fare defaults
- Installable, offline-capable PWA
- Every estimate still labelled as one

OUT OF SCOPE (FOR NOW)

- User accounts / login
- Live ERP or live traffic (no free feed)
- Real-time surge pricing (no Grab API)
- Any paid API or backend server
- Tracking / analytics that phone home

NON-NEGOTIABLE PRINCIPLE CARRIED FORWARD

The app’s trust proposition is “*we tell you what we assumed, and we don’t call anything home you didn’t expect.*” Every “auto” feature below is judged against that. Where automation needs an external call (routing, geocoding, map tiles), that is flagged as an explicit trade-off for you to accept or decline — never slipped in.

4 · WHERE WE ARE TODAY

The baseline (P0 — already shipped)

The app is deliberately split so that the part where a wrong answer is a *defect* is separated from the part where it is a *guess*. That split is the foundation everything below builds on.

FILE	LAYER	ROLE
`index.html`	Presentation	Markup, styles, all DOM wiring. The only file that touches the browser.
`calc.js`	Deterministic	Rules D1–D10. Pure functions, no DOM, unit-tested.
`geo.js` new	Deterministic	Haversine + winding-factor distance maths. Pure, unit-tested.
`data/*.json`	Assumptions	Every heuristic constant — parking (H1), fares (H2–H7), landmarks (H9).
`defaults.js`	Assumptions	Inline mirror + loader; keeps the app alive if a fetch fails.
`test/*.test.js`	Verification	60 unit tests, zero dependencies. Drift tests guard the mirror.

◆ JUST SHIPPED IN PR #1 — THE FIRST “AUTO” STEP

Use my location. Reads the phone’s GPS via the Web Geolocation API (iOS + Android), pairs it with a built-in Singapore landmark, and fills the one figure the user has to know — the one-way distance — via the pure maths in `geo.js`. It is a labelled estimate (straight-line × 1.35), stays editable, and degrades to plain typing if permission is denied. This plan is the roadmap for everything *after* that.

5 · TARGET ARCHITECTURE

The free auto web app, in layers

The shape does not change — it deepens. The deterministic core and the honest-assumptions layer stay exactly as they are. The “auto” work adds two things: **input sources** that feed the existing fields, and an **app shell** (PWA) that wraps the whole thing. Nothing new is allowed to reach the arithmetic except as a normal, labelled input.

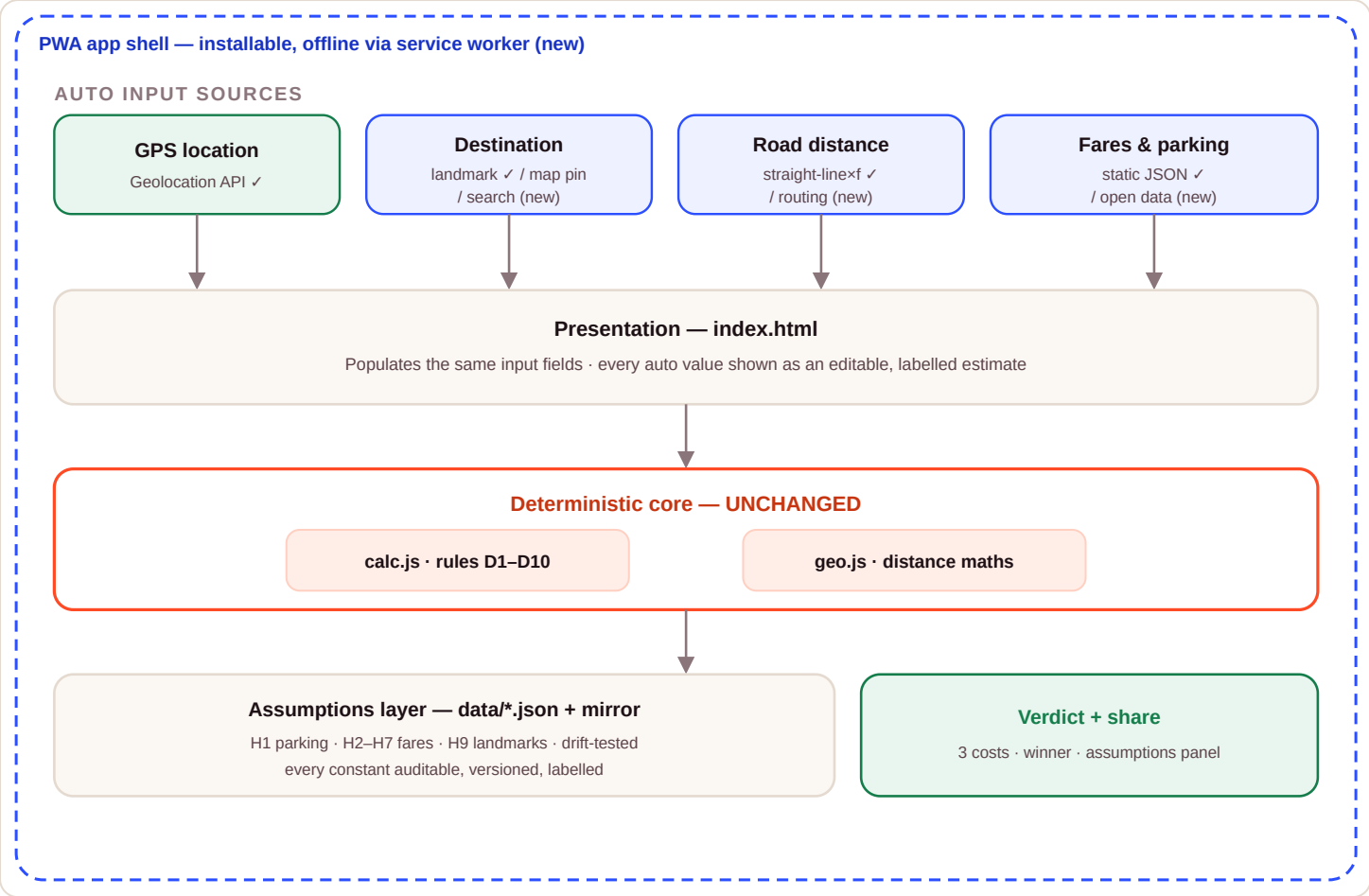


Figure 1 — Target architecture. ✓ = already shipped. Green = auto sources; red = the untouchable deterministic core.

6 · COMPONENT & DATA FLOW

What each new piece does

COMPONENT	NEW?	RESPONSIBILITY	FEEDS
Location detector	Done	GPS → origin coordinates	Road distance
Destination picker	Extend	Landmark list ✓, plus optional map pin / text search	Road distance
Distance provider	Extend	Straight-line×factor ✓, optionally swap in a real routing result	`distance` field
Open-data loader	New	Fetch & cache HDB/URA parking + LTA fares; fall back to today’s static JSON	Parking & fare fields
App shell (PWA)	New	Manifest + service worker: installable, offline, cached assets	Whole app
(Optional) Assist layer	Decision	Natural-language trip parsing — see Section 7	All fields

The happy-path flow, end to end

1. User opens the installed app at the door. Service worker serves it instantly, even offline.

2. Tap **Use my location** → origin coordinates (on-device, never sent anywhere).
3. Pick / search / pin the destination → destination coordinates.
4. Distance provider returns the one-way km — labelled estimate, editable.
5. Parking rate & fares pre-filled from cached open data (or static fallback) — each labelled with its source.
6. User confirms “how long will you park?” — often the *only* thing they type.
7. Deterministic core computes the three costs; verdict + assumptions panel render, unchanged.

7 · THE “AGENT” QUESTION — THE PIVOTAL CHOICE

Automation, autonomy, or AI?

This is the decision the rest of the build hangs on, so it gets its own section. The repo’s own design notes (`docs/DECISIONS.md`) already assessed and *rejected* a full agent: “this problem is not orchestration-driven… `parkRate × parkHours` is not an inference.” That finding still holds — but “free auto web app agent” may mean something lighter, so here are the three honest options.

OPTION A — SMART AUTOMATIC APP · NO AI (RECOMMENDED)

“Agent” = the app does the legwork automatically: senses location, derives distance, pulls open data, pre-fills everything. Pure deterministic code. **Free, private, offline-capable, testable, and fully in keeping with the existing design.** This delivers the entire “auto” goal with zero new trust cost.

OPTION B — THIN AI ASSIST, FREE & PRIVATE

Add one narrow AI touch: type or speak “*taking mum to SGH for about 3 hours Saturday*” and it fills the fields. Kept free & private by running a small model **in the browser** (WebLLM / transformers.js via WebGPU) — no server, no API key, nothing leaves the device. Cost: a one-time model download (tens–hundreds of MB) and the arithmetic is still deterministic underneath. The AI only parses input; it never computes the answer.

OPTION C — FULL LLM AGENT

A model plans and orchestrates the whole interaction. **Not recommended here:** to stay truly free it needs either a user-supplied API key or a hosted free tier (rate-limited, can vanish), it breaks the “calls nothing home” promise, it can’t be unit-tested, and it adds inference where there is no judgement to delegate. High cost, low fit.

DIMENSION	A · SMART AUTO	B · THIN IN-BROWSER AI	C · FULL AGENT
Truly free	Yes	Yes	Only via user key / fragile free tier
Private (nothing leaves device)	Yes	Yes	No
Works offline	Yes	After model cached	No
Unit-testable core	Yes	Yes (AI only parses)	No
Fits existing design	Perfectly	Mostly	Conflicts
Build effort	Low–medium	Medium–high	High

◆ DECISION NEEDED

My recommendation is **A**, with **B** as an optional later flourish if you want the natural-language input. Please confirm your reading of “agent” — it is Decision 1 in Section 10.

8 · THE “AUTO” UPGRADES IN DETAIL

Four ways the app fills itself in

8.1 · Real road distance

Today’s distance is straight-line × 1.35 — free, offline, but only an estimate. A real routed distance is more accurate but needs an external routing service.

OPTION	COST	TRADE-OFF
Keep straight-line × factor ✓	Free	Offline & private, but ~10–30% off real roads. Labelled honestly.
OpenRouteService (free API key)	Free tier	Accurate; ~2,000 routes/day free; one external call, needs a key.
OSRM public demo server	Free	No key; not guaranteed for production use — best as enhancement, not dependency.

Recommended: keep straight-line as the always-works default; optionally layer a routing call on top when online, clearly marked. Decision 3.

8.2 · Destination selection

The landmark dropdown ✓ covers common trips. Two optional additions widen it: a **map pin** (Leaflet + OpenStreetMap tiles) and **text search** (Nominatim geocoding). Both are free under fair-use policies but add an external call and map-tile loads.

Recommended: keep the landmark list as the zero-dependency core; add a map pin as a progressive enhancement. Decision 4.

8.3 · Fares & parking from open data

This is the highest-value upgrade the repo already names: parking rates are a guess only because the data isn’t loaded, and **Singapore publishes it**. The plan keeps today’s static JSON as the guaranteed fallback and layers real data on top.

DATA	FREE SOURCE	NOTES
HDB carpark info	data.gov.sg (open)	Rates & locations; the H1 upgrade path already written in the repo.
URA / private carparks	URA API (free key) / data.gov.sg	Central-area & season rates.
MRT / bus fares	LTA DataMall / PTC fare tables	Per your note: *we don’t estimate MRT — you enter it from the transit app.* Open fare tables can pre-fill a sensible default only.

Recommended: build a small loader that fetches & caches this, always falling back to the static mirror. Exact endpoints to be verified in P2. Decision 5.

8.4 · Make it an installable app (PWA)

A web manifest + service worker turns the page into an app you add to your home screen, with an icon, and that opens instantly and works offline. No frameworks, no build step — a few small files. This is what makes it a “web app” rather than a web page. Decision 6.

9 · THE FREE-TIER STACK

Proof that every layer is $0

LAYER	CHOICE	COST	LIMIT / CAVEAT
Hosting	GitHub Pages	Free	Already in use; static only (which is all we need).
Build	None	Free	No bundler, no transpiler — a deliberate constraint.
Location	Web Geolocation API	Free	Browser capability; needs HTTPS (Pages provides it).
Distance	geo.js / OpenRouteService	Free	Local maths free; routing free-tier ~2k/day.
Map (optional)	Leaflet + OSM tiles	Free	Fair-use tile policy; keep usage light.
Parking / fares	data.gov.sg / LTA / URA	Free	Open data; free API keys where needed.
AI (if Option B)	In-browser WebLLM	Free	One-time model download; runs on device.
Analytics	None (or Plausible free)	Free	Kept off by default — respects the trust model.

THE HONEST ASTERISK ON “FREE FOREVER”

Free *tiers* and fair-use policies can change or rate-limit. The design defends against this by making every external source **optional and cached, with a static fallback**: if a free tier disappears, the app degrades to today’s labelled estimates rather than breaking. That is the same parachute pattern the code already uses for `data/*.json`.

10 · PHASED ROADMAP

Order of work

PHASE	SCOPE	EXIT CRITERIA	STATUS
P0	Location detection → auto distance (landmark + straight-line)	Shipped in PR #1; 60 tests green	Done
P1	PWA shell (manifest + service worker, installable, offline)	Installs on iOS/Android; works offline	Next
P2	Open-data loader for parking (H1) + fare defaults, with fallback	Real rates when online; static mirror when not; drift-tested	Planned
P3	Optional map pin + routing distance (progressive enhancement)	Accurate distance online; landmark+straight-line offline	Planned
P4?	Optional Option-B in-browser AI trip parsing	Only if you choose Option B in Section 7	If wanted

11 · RISKS & MITIGATIONS

What could go wrong, and the answer

RISK	MITIGATION
A free tier rate-limits or shuts down	Every external source is optional + cached, with the static JSON as fallback. App degrades, never breaks.
Automation makes a wrong guess look authoritative	Every auto value is labelled an estimate, shows its source, and stays editable — the existing discipline, extended.
Adding external calls erodes the “calls nothing home” trust	Each is a flagged, opt-in decision (Section 10); location data never leaves the device; no tracking.
PWA offline cache serves stale data	Service worker uses a versioned cache; assumptions carry a version stamp already.
Scope creep toward a full agent	This document fixes the boundary: deterministic core is untouchable; AI (if any) only parses input.

12 · TESTING STRATEGY

How each layer is proven

- **Deterministic layers** (`calc.js`, `geo.js`, future distance/data maths) → Node unit tests, zero dependencies. A wrong answer here is a defect.
- **Assumption files** → drift tests keep JSON and mirror identical; structural tests keep them usable (coords in Singapore, rates non-negative, every heuristic explains itself).
- **Heuristics** (winding factor, guessed rates) → not unit-tested — labelled in the UI instead. You cannot unit-test a guess.
- **UI & integration** → headless Chromium checks (as used to verify PR #1: happy path, geometry sanity, permission-denied fallback).

- **CI gate** → GitHub Actions runs the suite on every push; deploy only if green.

Nine questions, each with a default

This is the heart of the sign-off. If the defaults look right, “go with the defaults” is a complete answer and I’ll implement P1 next. Otherwise, mark up the ones you want to change.

1 What does “agent” mean to you?

Default: Option A — a smart automatic app, no AI. (Option B, free in-browser AI, available later if you want it.)

2 Is the scope in Section 3 right — anything to add or drop?

Default: Scope as written; accounts, live ERP, surge pricing stay out.

3 Real road distance: add a routing call, or keep straight-line only?

Default: Keep straight-line as the always-works core; add optional routing as an online enhancement.

4 Destination input: landmark list only, or also a map pin / search?

Default: Keep landmark list; add a map pin in P3 as progressive enhancement.

5 Open-data parking & fares: worth building the loader?

Default: Yes — it’s the highest-value upgrade; always with static fallback. MRT fare stays user-entered per your note.

6 Make it an installable PWA (P1)?

Default: Yes — this is what makes it a “web app,” and it’s cheap to build.

7 Any external call at all, or strictly zero-dependency?

Default: External calls allowed only when opt-in, cached, and with a fallback; location never leaves the device.

8 Phase order — start with P1 (PWA) as proposed?

Default: P1 → P2 → P3, with P4 (AI) only if Decision 1 is Option B.

9 Analytics — keep it fully off?

Default: Off. Add privacy-respecting Plausible only if you later want usage numbers.

DEFINITION OF DONE FOR THE WHOLE EFFORT

A user at the door opens the installed app, taps once, picks a destination, and gets an honest three-way cost verdict having typed almost nothing — for free, privately, and with every assumption still visible. That is the finish line.